

Project 1: Reverse engineering, Analysis and Refactoring of Apache Roller

CS6.401 – Software Engineering

TEAM 33

Members

- Aryan Mishra, 2024121001
 - Aditya Nair, 2023111029
 - Varun Patil, 2025204013
 - Hardik Chadha, 2023111031
 - Vijay Aravynthan, 2023111007
-

Task 1: Architectural Mapping and Design Recovery

1. Weblog and Content Subsystem

Overview

The weblog and content subsystem is responsible for managing blogs, blog entries, comments, categories, and related content. It encapsulates both the data representation of weblog content and the business logic for creating, updating, and retrieving this content.

Key Classes and Interfaces

1. WeblogManager (Interface)

Role:

WeblogManager defines the service-level contract for managing weblogs and retrieving their associated content.

Responsibilities:

- Persisting weblog objects
- Fetching weblogs using unique identifiers (handles)
- Retrieving entries belonging to a weblog

Key Methods:

- `saveWeblog(Weblog)`
- `getWeblogByHandle(String)`
- `getWeblogEntries(Weblog)`

Interactions:

- Implemented by `JPAWeblogManagerImpl`
 - Operates on `Weblog` domain objects
 - Acts as an abstraction between higher layers and persistence logic
-

2. JPAWeblogManagerImpl (Class)

Role:

Concrete implementation of `WeblogManager` using JPA for persistence.

Responsibilities:

- Implements all weblog-related operations
- Coordinates persistence of weblog entities
- Acts as the primary access point for weblog storage and retrieval

Key Methods:

- `saveWeblog(Weblog)`
- `getWeblogByHandle(String)`
- `getWeblogEntries(Weblog)`

Interactions:

- Implements `WeblogManager`
 - Directly interacts with `Weblog` and `WeblogEntry`
 - Serves as a bridge between domain logic and the persistence layer
-

3. Weblog (Class)

Role:

Represents a blog (weblog) owned by a user or group.

Attributes:

- `id`, `handle`, `name`
- `description`, `locale`, `timeZone`

Responsibilities:

- Stores metadata related to a weblog
- Acts as the aggregate root for content entities

Key Methods:

- `getHandle()`
- `getName()`
- `getDescription()`

Relationships:

- One-to-many association with:
 - `WeblogCategory`
 - `WeblogBookmark`
 - `WeblogEntry`
-

4. WeblogEntryManager (Interface)

Role:

Defines operations for managing weblog entries.

Responsibilities:

- Creating and removing blog entries
- Fetching individual entries

Key Methods:

- `saveWeblogEntry(WeblogEntry)`
- `removeWeblogEntry(WeblogEntry)`
- `getWeblogEntry(String)`

Interactions:

- Operates on `WeblogEntry`
 - Implementations coordinate entry lifecycle management
-

5. WeblogEntry (Class)

Role:

Represents an individual blog post within a weblog.

Attributes:

- `id, title, text`
- `pubTime, status`
- `allowComments`

Responsibilities:

- Stores content and publication metadata
- Determines publication state

Key Methods:

- `isPublished()`
- `getTitle()`
- `getPubTime()`

Relationships:

- Many-to-one association with `Weblog`
 - One-to-many composition with `WeblogEntryComment`
-

6. `WeblogEntryComment` (Class)

Role:

Represents a comment made on a weblog entry.

Attributes:

- `id, content`
- `postTime, approved`

Responsibilities:

- Stores comment content and moderation state

Key Methods:

- `isApproved()`
- `getPostTime()`

Relationships:

- Belongs to a single `WeblogEntry`
-

7. `WeblogEntryRenderer` (Class)

Role:

Responsible for rendering weblog entries into a presentable format (e.g., HTML).

Responsibilities:

- Transforms raw entry data into rendered output

Key Methods:

- `render(WeblogEntry)`

Interactions:

- Depends on `WeblogEntry`
 - Typically invoked during content delivery
-

8. WeblogCategory (Class)

Role:

Represents a category used to organize weblog entries.

Attributes:

- `id, name, description`

Responsibilities:

- Provides logical grouping for weblog entries

Key Methods:

- `getName()`

Relationships:

- Associated with a `Weblog`
-

9. WeblogBookmark (Class)

Role:

Represents a bookmarked external link associated with a weblog.

Attributes:

- `id, name`
- `url, description`

Responsibilities:

- Stores user-curated external references

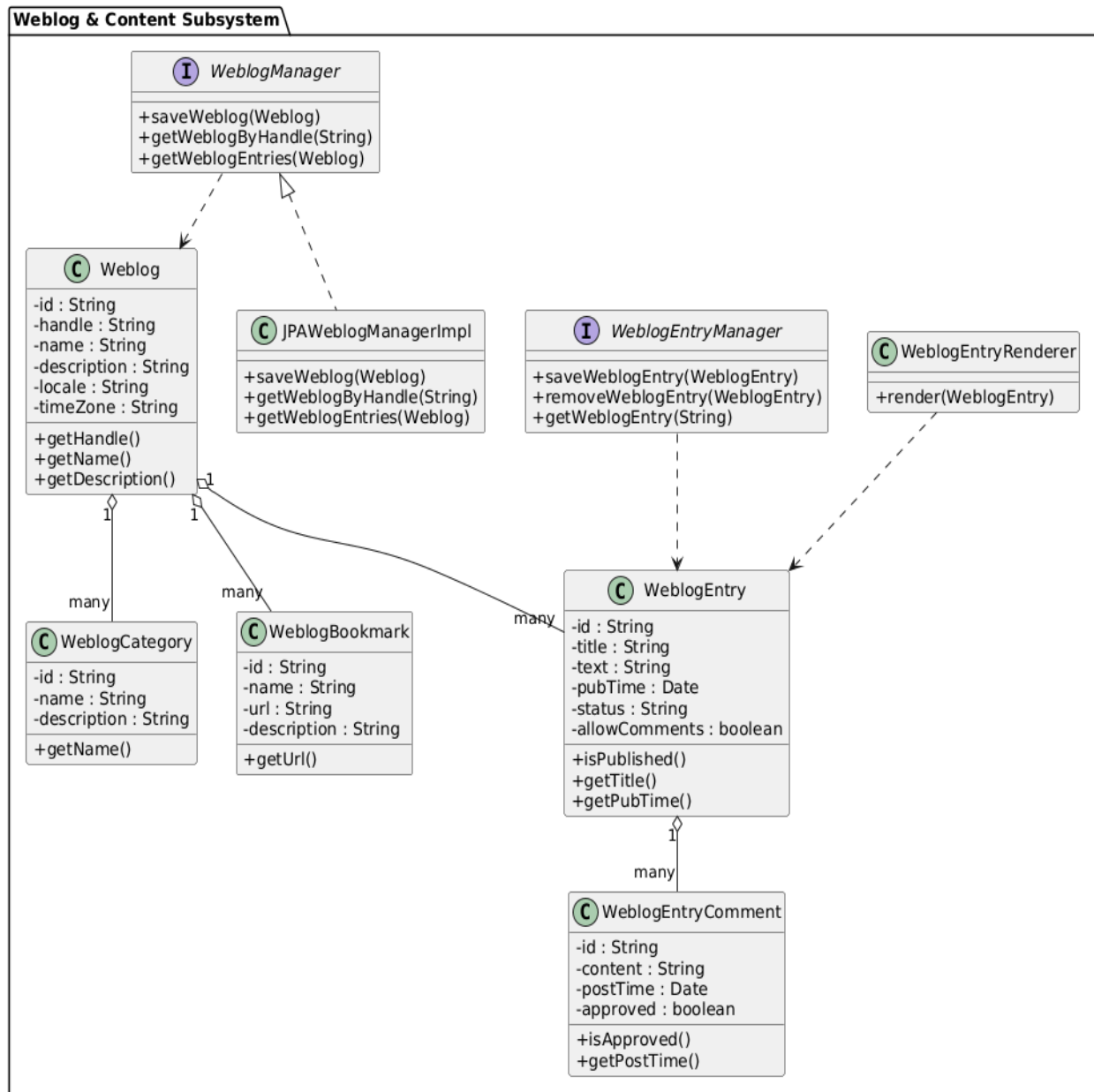
Key Methods:

- `getUrl()`

Relationships:

- Associated with a `Weblog`
-

UML Diagram:



The UML class diagram for this subsystem models:

- Interface implementation (**WeblogManager** → **JPAWeblogManagerImpl**)
- Aggregation and composition relationships between weblogs, entries, comments, categories, and bookmarks
- Dependency relationships between manager and renderer components and domain entities

UML Modeling Assumptions

- Only core content-related classes were modeled.
 - UI controllers and persistence details were abstracted.
 - Attributes shown are representative, not exhaustive.
 - Only architecturally significant attributes and public methods were modeled. Getters, setters, persistence helpers, and utility methods were omitted to maintain clarity and focus on design-level relationships.
 -
-

Observations and Comments

Strengths

- Clear separation between domain entities and service interfaces. This improves modularity and makes the system easier to scale.
- Use of Manager interfaces (`WeblogManager`, `WeblogEntryManager`) improves testability.
- Aggregation structure around `Weblog` is intuitive and domain-aligned

Weaknesses

- Manager implementations (e.g., `JPAWeblogManagerImpl`) risk becoming overly large, which can lead to God-class tendencies.
 - Limited Encapsulation of Rendering Logic. It is separated but tightly coupled to domain/content objects
 - Persistence concerns are closely intertwined with business logic
-

2. User and Role Management Subsystem

Overview

The User and Role Management subsystem plays a critical role in the weblogger application by supporting its security and personalization features. It is responsible for user authentication, role assignment, permission verification, and access control for both system-wide operations and weblog-specific actions.

The subsystem is designed using a layered architecture that clearly separates responsibilities across different tiers. The Presentation Layer handles user interactions, the Business Layer manages application logic, and the Domain Layer represents core entities. This separation of concerns improves modularity, maintainability, and clarity of the overall system design.

Architectural Design

The architecture follows a standard, pattern-based structure commonly used in enterprise Java applications.

1. **Presentation Layer**

The Presentation Layer consists of Struts 2 Action classes such as `Register` and `Members`. These classes are responsible for handling HTTP requests, validating user input, and coordinating user interactions. They do not contain business logic but delegate such responsibilities to the service layer.

2. **Service Layer**

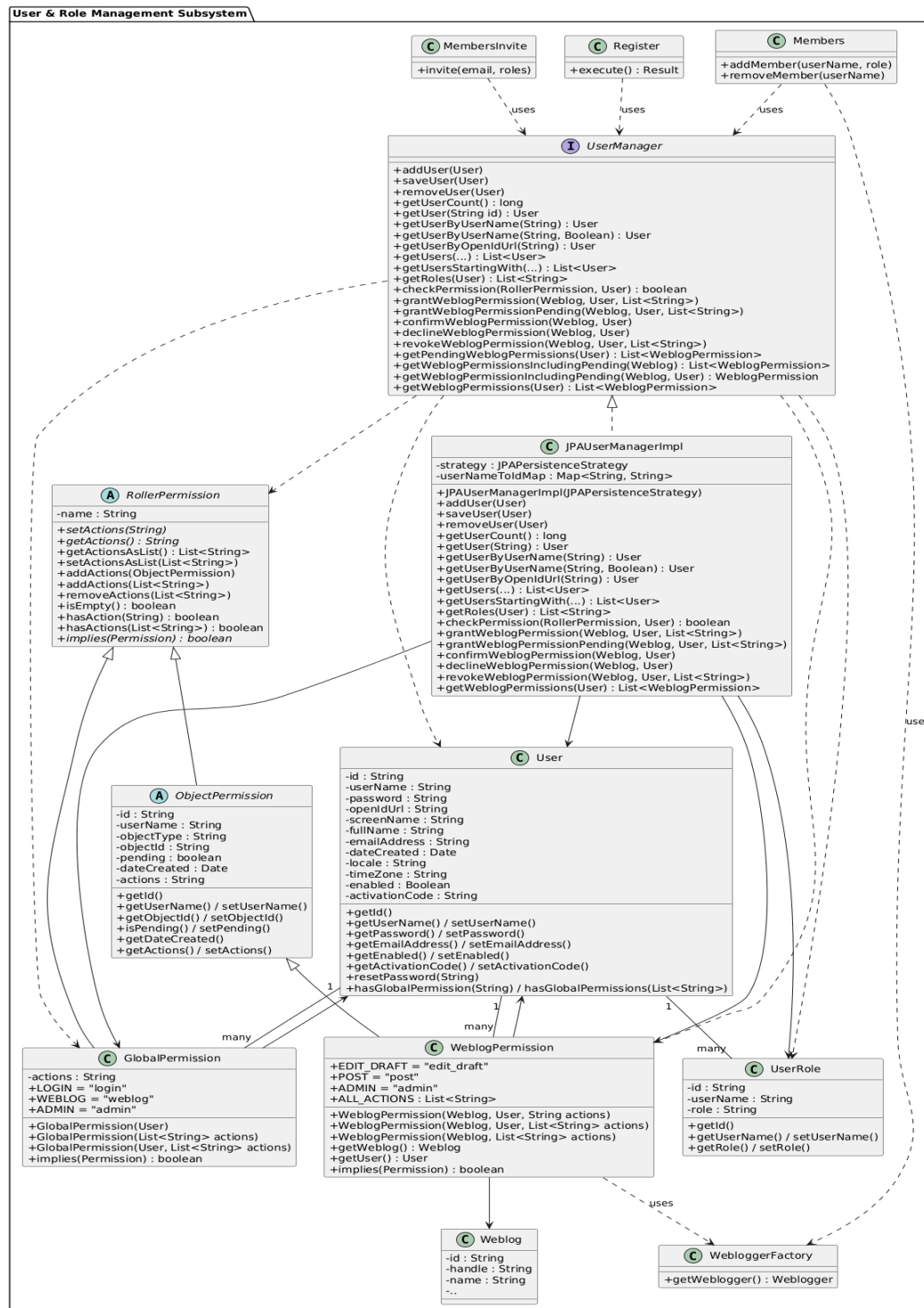
The Service Layer is defined by the `UserManager` interface, which specifies the business operations related to user and role management. This interface abstracts away the details of data storage and persistence. The actual persistence logic is implemented in `JPAUserManagerImpl`, which uses JPA to interact with the database. This design allows the rest of the application to remain independent of the underlying persistence technology.

3. **Domain Layer**

The Domain Layer consists of Plain Old Java Objects (POJOs) such as `User` and `RollerPermission`. These classes encapsulate the core state and behavior of users and their associated permissions. By modeling permissions as domain objects, the system enables flexible and expressive authorization logic.

Class Diagram

The accompanying class diagram illustrates the structural relationships between the key components of the subsystem. The classes are grouped according to their architectural layers and packages, highlighting the interaction between presentation components, service interfaces, and domain entities.



Key Classes

This section outlines the major classes involved in the authorization and user management subsystem. The classes are grouped based on their roles in the system: domain representation, permission rules, business logic, and presentation-level interaction.

1. Domain Classes (The Core Entities)

User (org.apache.roller.weblogger.pojos)

Responsibility: Represents a user account within the system. The `User` class stores authentication credentials and profile-related information and serves as the primary representation of user identity.

Key Attributes:

- `userName`
- `password`
- `emailAddress`
- `fullName`
- `enabled`
- `activationCode`

Collaborators:

- `UserManager`
- `UserRole`

UserRole (org.apache.roller.weblogger.pojos)

Responsibility: Represents a role assigned to a user, such as “admin” or “editor”. This class associates a user with a high-level role that determines the permissions available to them.

Key Attributes:

- `userName`
- `Role`

2. Permission Classes (Authorization Rules)

RollerPermission (Abstract Base Class)

Responsibility: Serves as the abstract base class for all permission types in the system. It extends Java's standard `java.security.Permission`, allowing the application to reuse established Java security mechanisms.

Capabilities:

Defines the general model for action-based access control and provides the foundation for permission implication logic.

3. GlobalPermission

Responsibility: Handles system-wide permissions that are not tied to a specific weblog instance.

Supported Actions:

- ADMIN: Full system access
- WEBLOG: Permission to create weblogs
- LOGIN: Basic login access

Logic:

Implements hierarchical permission logic where higher-level permissions imply lower-level ones (for example, ADMIN implies both WEBLOG and LOGIN).

WeblogPermission

Responsibility: Manages permissions that apply to a specific weblog.

Supported Actions:

- ADMIN: Full control over the weblog (typically the owner)
- POST: Permission to publish entries (editor role)
- EDIT_DRAFT: Permission to create and edit drafts without publishing

3. Service Classes (Business Logic Layer)

UserManager (org.apache.roller.weblogger.business)

Responsibility: Acts as the central interface for managing users, roles, and permissions. It abstracts away persistence and storage concerns, exposing only business-level operations.

Key Methods:

- `addUser(User) / saveUser(User)`: Manage the lifecycle of user accounts
- `checkPermission(RollerPermission, User)`: Determine whether a user is authorized to perform a given action
- `grantWeblogPermission(...)`: Assign permissions to users for specific weblogs

4. Presentation Classes (Web Interaction Layer)

Register

Responsibility: Handles the user self-registration workflow. It validates user input (such as usernames and passwords) and interacts with `UserManager` to create new user accounts.

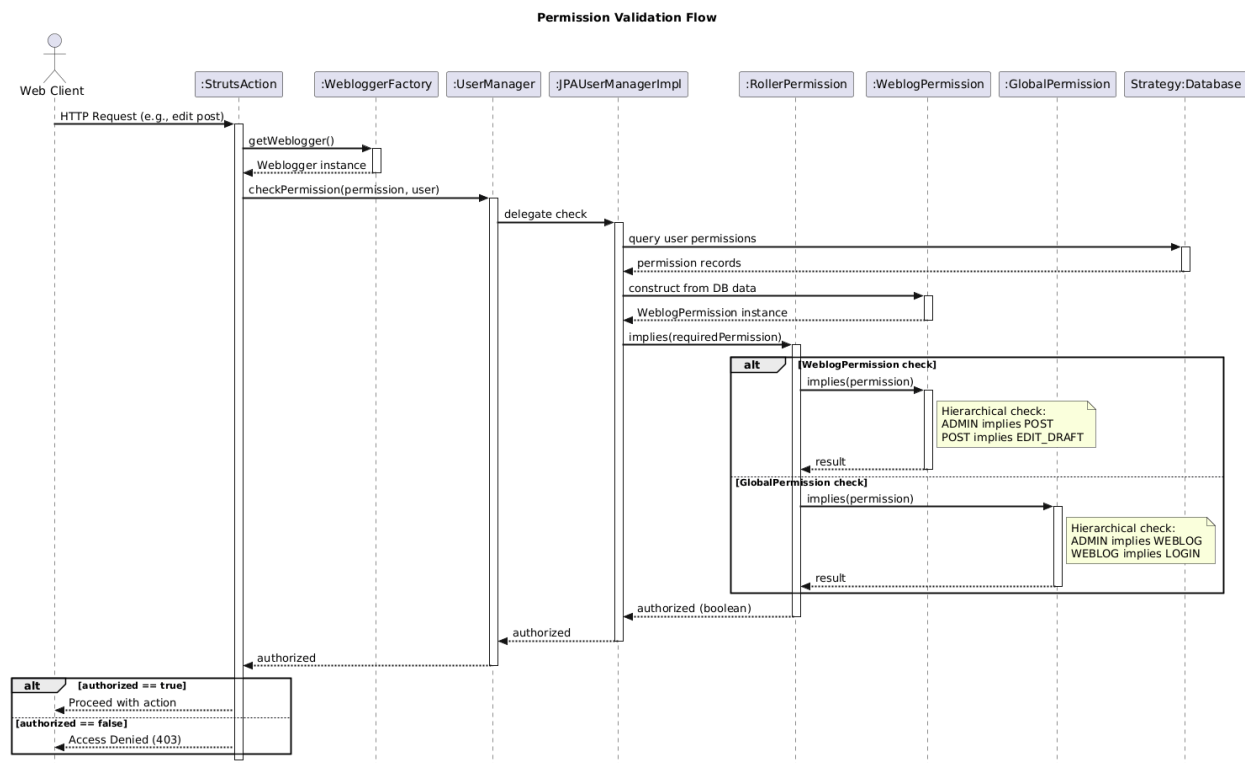
Members

Responsibility: Manages the “Team” or “Members” page for a specific weblog. It allows weblog owners to invite new members and modify the permissions of existing members.

System Behavior: Permission Validation Flow

The permission validation process follows a structured flow involving multiple layers of the application. A client request is first handled by a Struts Action, which delegates authorization checks to the `userManager`. The `userManager` then relies on its concrete implementation (`JPAUserManagerImpl`) to retrieve relevant data from the persistence layer. Permission objects are reconstructed from the database, and the `implies()` method is used to determine whether the requested action is authorized.

This layered interaction ensures a clear separation between presentation logic, business rules, and persistence concerns.



Design Analysis and Critique

This section combines insights from both manual code inspection and automated analysis to evaluate the architectural strengths and weaknesses of the system. The discussion highlights effective design choices as well as areas where architectural debt has accumulated.

Strengths

1. Hierarchical Permission Model (Smart Permissions)

The use of the `RollerPermission.implies()` mechanism is a strong design decision. Instead of explicitly enumerating every allowed action for each role, permissions are defined hierarchically (for example, ADMIN implies POST, which in turn implies EDIT_DRAFT). This approach simplifies authorization logic, reduces redundancy, and makes permission management easier to extend.

2. Effective Use of the Manager Pattern

The `UserManager` interface successfully abstracts the underlying JPA-based persistence logic from the presentation layer. This separation allows web actions and controllers to focus on application flow and business logic rather than database-specific concerns, improving modularity and maintainability.

3. Dual-Scope Authorization Model

The system differentiates between `GlobalPermission` (system-wide privileges) and `WeblogPermission` (resource-specific privileges). This distinction is well-suited for a multi-tenant architecture, where permissions must be applied at both global and per-resource levels.

Weaknesses and Architectural Debt

1. God Class Violation

The `JPAUserManagerImpl` class violates the Single Responsibility Principle by handling multiple unrelated concerns, including user persistence, permission checks, caching, and role management. This high level of responsibility increases coupling, reduces readability, and makes the class difficult to test and maintain.

2. Anemic Domain Model

The `User` class primarily acts as a data container with getters and setters, containing little to no domain behavior. In a more object-oriented design, the `User` entity would encapsulate authorization logic and actively participate in permission checks rather than being a passive object evaluated by manager classes.

3. String-Based Role Representation (Primitive Obsession)

User roles are represented as plain strings (such as "admin" or "editor"), which introduces several issues:

- Lack of compile-time validation for role names
- Implicit and fragile mappings defined in configuration files
- Increased risk of errors during refactoring or renaming of roles

Using explicit role types or enums would improve safety and clarity.

4. Static Coupling to Infrastructure

The `WeblogPermission` class depends directly on `WebloggerFactory` to locate managers. This creates tight coupling between domain objects and the service-layer factory, leading to circular dependencies and reducing flexibility. Such static coupling makes testing more difficult and limits the ability to evolve the architecture cleanly.

3. Search and Indexing Subsystem:

Overview

The search and indexing subsystem provides full-text search over weblog entries and comments. It uses a Lucene-backed index managed by `IndexManager` and its Lucene implementation. Search requests are initiated by HTTP servlets and feed handlers, translated into Lucene queries, and converted back into `SearchResultList` objects for rendering in the UI and feeds.

Key Classes and Interfaces

1. `IndexManager`

Package: `org.apache.roller.weblogger.business.search`

Responsibility: Defines the public search API for initialization, rebuilding indexes, adding/removing entries, executing searches, and managing lifecycle.

Key Methods:

- `void initialize()` - Initializes the search index and underlying Lucene infrastructure
- `void shutdown()` - Cleanly shuts down the index manager and releases all resources
- `void release()` - Releases resources associated with the current Roller session
- `boolean isInconsistentAtStartup()` - Returns whether the index was found inconsistent at startup (triggers automatic rebuild)
- `void rebuildWeblogIndex()` - Rebuilds the entire search index for all weblogs
- `void rebuildWeblogIndex(Weblog weblog)` - Rebuilds the search index for a specific weblog
- `void addEntryIndexOperation(WeblogEntry entry)` - Schedules an operation to add an entry to the index
- `void addEntryReIndexOperation(WeblogEntry entry)` - Schedules an operation to re-index an existing entry
- `void removeEntryIndexOperation(WeblogEntry entry)` - Schedules an operation to remove an entry from the index
- `void removeWeblogIndex(Weblog weblog)` - Removes all index entries for a specific weblog
- `SearchResultList search(String term, String weblogHandle, String category, String locale, int pageNum, int entryCount, URLStrategy urlStrategy)` - Executes a search query and returns paginated results

Collaborates With: `LuceneIndexManager`, `SearchResultList`, `URLStrategy`, `WeblogEntry`, `Weblog`

2. LuceneIndexManager

Package: `org.apache.roller.weblogger.business.search.lucene`

Responsibility: Lucene-based implementation of `IndexManager`. Manages index lifecycle, schedules index operations, executes search queries, and converts Lucene hits to domain-level wrappers. Handles concurrency through a `ReadWriteLock` and maintains a shared `IndexReader` for search operations.

Key Attributes/Methods:

- `IndexReader reader` - Shared reader for all search operations (reset after write operations)
- `Weblogger roller` - Reference to `Weblogger` instance for service access
- `boolean searchEnabled` - Feature toggle from configuration
- `String indexDir` - File system path to the Lucene index directory
- `File indexConsistencyMarker` - Marker file to detect index corruption or version incompatibility
- `boolean inconsistentAtStartup` - Flag indicating whether index was found inconsistent at initialization
- `ReadWriteLock rwl` - Concurrency control for index access (read operations share lock, writes are exclusive)
- `initialize()` - Initializes Lucene directory, checks index consistency, creates marker file
- `search(...)` - Executes `SearchOperation` and converts Lucene hits to `WeblogEntryWrapper` list
- `scheduleIndexOperation(IndexOperation)` - Submits operation to `ThreadManager` for background execution
- `executeIndexOperationNow(IndexOperation)` - Executes operation synchronously in foreground
- `getSharedIndexReader()` - Returns the shared reader (lazily initialized)
- `resetSharedReader()` - Closes and nulls the shared reader (called after write operations)
- `getAnalyzer()` - Returns configured Lucene analyzer for text processing

Concurrency Model:

Read operations (searches) acquire a read lock and share the `IndexReader`. Write operations (add/remove/rebuild) acquire a write lock, perform modifications through `IndexWriter`, and reset the shared reader to ensure subsequent searches see the updates. Background operations are scheduled through `ThreadManager` while synchronous removals use `executeIndexOperationNow()` to ensure immediate consistency.

Collaborates With: `IndexOperation` hierarchy, `WeblogEntryManager`, `URLStrategy`, `WebloggerFactory`, `ThreadManager`, `WebloggerConfig`

3. IndexOperation (abstract)

Package: `org.apache.roller.weblogger.business.search.lucene`

Responsibility: Base class for all index operations; builds Lucene `Document` objects for `WeblogEntry` instances and manages `IndexWriter` lifecycle.

Key Attributes/Methods:

- `LuceneIndexManager manager`

- `IndexWriter writer`
- `getDocument(WeblogEntry)`
- `beginWriting(), endWriting()`
- `doRun()` (template method)

Collaborates With: `LuceneIndexManager`, `FieldConstants`, `WeblogEntry`, `WeblogEntryComment`

4. ReadFromIndexOperation / WriteToIndexOperation

Package: `org.apache.roller.weblogger.business.search.lucene`

Responsibility: Concurrency wrappers that execute index operations under read/write locks. `WriteToIndexOperation` also resets the shared reader after writes.

Key Attributes/Methods:

- `run()` (lock acquisition + `doRun()`)
- Collaborates With:** `LuceneIndexManager`

5. SearchOperation

Package: `org.apache.roller.weblogger.business.search.lucene`

Responsibility: Builds Lucene query objects and executes searches against the index. Supports filters by weblog handle, category, and locale. Uses `MultiFieldQueryParser` to search across multiple content fields and returns results sorted by publication date.

Key Attributes/Methods:

- `String term` - Search query term
- `String weblogHandle` - Optional filter for specific weblog
- `String category` - Optional filter for category
- `String locale` - Optional filter for content locale
- `IndexSearcher searcher` - Lucene searcher instance used for query execution
- `TopFieldDocs searchresults` - Lucene search results containing scored documents
- `String parseError` - Error message if query parsing fails
- `static final String[] SEARCH_FIELDS = {FieldConstants.CONTENT, FieldConstants.TITLE, FieldConstants.C_CONTENT}` - Fields searched by default (entry content, title, and comment content)
- `static final Sort SORTER` - Sort configuration (by `PUBLISHED` field, descending order)
- `static final int docLimit = 500` - Hard-coded maximum number of documents to return
- `doRun()` - Builds query, applies filters, executes search
- `getResults()` - Returns `TopFieldDocs` result set
- `getResultsCount()` - Returns total hit count

Collaborates With: `IndexUtil`, `FieldConstants`, `LuceneIndexManager`, `MultiFieldQueryParser`

6. `AddEntryOperation` / `RemoveEntryOperation` / `ReIndexEntryOperation`

Package: `org.apache.roller.weblogger.business.search.lucene`

Responsibility: Index mutations for individual entries (add, remove, reindex). They re-fetch entries through `WeblogEntryManager` to avoid detached entity issues.

Collaborates With: `WeblogEntryManager`, `Weblogger`, `LuceneIndexManager`

7. `RebuildWebsiteIndexOperation` / `RemoveWebsiteIndexOperation`

Package: `org.apache.roller.weblogger.business.search.lucene`

Responsibility: Bulk index operations to rebuild or remove a site's index, optionally for all sites.

Collaborates With: `WeblogManager`, `WeblogEntryManager`, `LuceneIndexManager`

8. `FieldConstants`

Package: `org.apache.roller.weblogger.business.search.lucene`

Responsibility: Defines constant field names for the Lucene document schema. These constants ensure consistent field naming across all index operations and searches.

Key Constants:

- `CONTENT` - Main weblog entry content field (searched, stored)
- `TITLE` - Entry title field (searched, stored)
- `C_CONTENT` - Comment content field (searched for comment searches)
- `PUBLISHED` - Publication timestamp field (used for sorting results)
- `HANDLE` - Weblog handle field (used for filtering by weblog)
- `LOCALE` - Content locale field (used for locale-specific filtering)
- `CATEGORY` - Entry category field (used for category filtering)
- `ID` - Unique entry identifier

Collaborates With: All `IndexOperation` subclasses, `SearchOperation`, `IndexUtil`

9. `IndexUtil`

Package: `org.apache.roller.weblogger.business.search.lucene`

Responsibility: Provides utility methods for query parsing, term construction, and field manipulation in Lucene queries.

Key Methods:

- Query parsing helpers for constructing `BooleanQuery` filters
- Term construction utilities for field-specific queries
- Field value extraction from Lucene Documents

Collaborates With: `SearchOperation`, `IndexOperation`, `FieldConstants`

10. SearchResultList / SearchResultMap

Package: `org.apache.roller.weblogger.business.search`

Responsibility: Simple data holders for search results. `SearchResultList` stores ordered results + category facets; `SearchResultMap` stores grouped results by date.

Collaborates With: `WeblogEntryWrapper`

11. WeblogSearchRequest

Package: `org.apache.roller.weblogger.ui.rendering.util`

Responsibility: Parses and validates search query parameters (query, page, category) for the search servlet. Lazily resolves `WeblogCategory` from the `WeblogEntryManager`.

Key Attributes/Methods:

- `String query` - The search query term
- `int pageNum` - Requested page number for pagination
- `String weblogCategoryName` - Category name filter (optional)
- `WeblogCategory weblogCategory` - Resolved category object (lazy-loaded)
- `getQuery(), getPageNum(), getWeblogCategory()`

Collaborates With: `WeblogEntryManager`, `WebloggerFactory`, `URLUtilities`

12. WeblogFeedRequest

Package: `org.apache.roller.weblogger.ui.rendering.util`

Responsibility: Parses and validates search and feed parameters for Atom/RSS feed-based search requests. Handles feed-specific parameters like entry excerpts, tags, and feed format while supporting search term filtering.

Key Attributes/Methods:

- `String term` - Search query term for feed filtering
- `int page` - Page number for feed pagination
- `String weblogCategoryName` - Category filter name
- `WeblogCategory weblogCategory` - Resolved category object
- `String type` - Feed type (entries, comments)
- `String format` - Feed format (atom, rss)
- `List<String> tags` - Tag filters for feed entries
- `boolean excerpts` - Whether to include full content or excerpts only
- `getTerm(), getPage(), getWeblogCategory(), getFormat(), getTags(), isExcerpts()`

Collaborates With: `WeblogEntryManager`, `WebloggerFactory`, `SearchResultsFeedModel`, `SearchResultsFeedPager`

13. SearchResultsModel

Package: `org.apache.roller.weblogger.ui.rendering.model`

Responsibility: Executes searches for HTML pages, groups results by date (midnight timestamps), and exposes pager/counters to templates. Extends `PageModel` to integrate with the rendering framework.

Key Attributes/Methods:

- `Map<Date, Set<WeblogEntryWrapper>> results` - Results grouped by publication date (truncated to midnight)
- `SearchResultsPager pager` - Pagination helper for building navigation links
- `int hits` - Total number of search hits
- `int offset` - Starting offset for current page
- `int limit` - Number of results per page
- `String errorMessage` - Error message if search fails
- `static final int RESULTS_PER_PAGE = 10` - Page size constant
- `init(Map initData)` - Initializes model with request context and executes search
- `getResults()` - Returns date-grouped results map
- `addEntryToResults(WeblogEntryWrapper)` - Helper to add entry to date-grouped structure

Collaborates With: `IndexManager`, `SearchResultList`, `SearchResultsPager`, `WeblogSearchRequest`, `PageModel` (superclass)

14. SearchResultsFeedModel

Package: `org.apache.roller.weblogger.ui.rendering.model`

Responsibility: Executes searches for Atom/RSS feeds and exposes results and pagination metadata. Implements the `Model` interface to integrate with the feed rendering framework.

Key Attributes/Methods:

- `List<WeblogEntryWrapper> results` - Linear list of search results for feed serialization
- `SearchResultsFeedPager pager` - Feed-specific pagination helper
- `int hits` - Total number of search hits
- `int offset` - Starting offset for current page
- `int limit` - Number of results per page (from configuration)
- `String errorMessage` - Error message if search fails
- `init(Map initData)` - Initializes model with feed request context and executes search
- `getResults()` - Returns linear results list
- `getModelName()` - Returns model identifier for template access (required by `Model` interface)

Collaborates With: `IndexManager`, `SearchResultList`, `SearchResultsFeedPager`, `WeblogFeedRequest`, `Model` (interface), `WebloggerRuntimeConfig`

15. SearchResultsPager / SearchResultsFeedPager

Package: `org.apache.roller.weblogger.ui.rendering.pagers`

Responsibility: Builds navigation links for search pages and search feeds, preserving query/category parameters.

Key Attributes/Methods:

- `String query, category, page` (search page)
- `WeblogFeedRequest feedRequest` (feed pager)
- `getNextLink(), getPrevLink(), getUrl()`

Collaborates With: `URLStrategy, WeblogSearchRequest, WeblogFeedRequest`

16. SearchServlet

Package: `org.apache.roller.weblogger.ui.rendering.servlets`

Responsibility: HTTP entry point for search pages. Parses `WeblogSearchRequest`, loads models for rendering, and serves the search page template.

Collaborates With: `ModelLoader, WebloggerFactory, ThemeManager, RendererManager`

17. OpenSearchServlet

Package: `org.apache.roller.weblogger.webservices.opensearch`

Responsibility: Serves OpenSearch descriptor XML to enable external clients to discover search endpoints.

Collaborates With: `URLStrategy, WebloggerFactory, WebloggerRuntimeConfig`

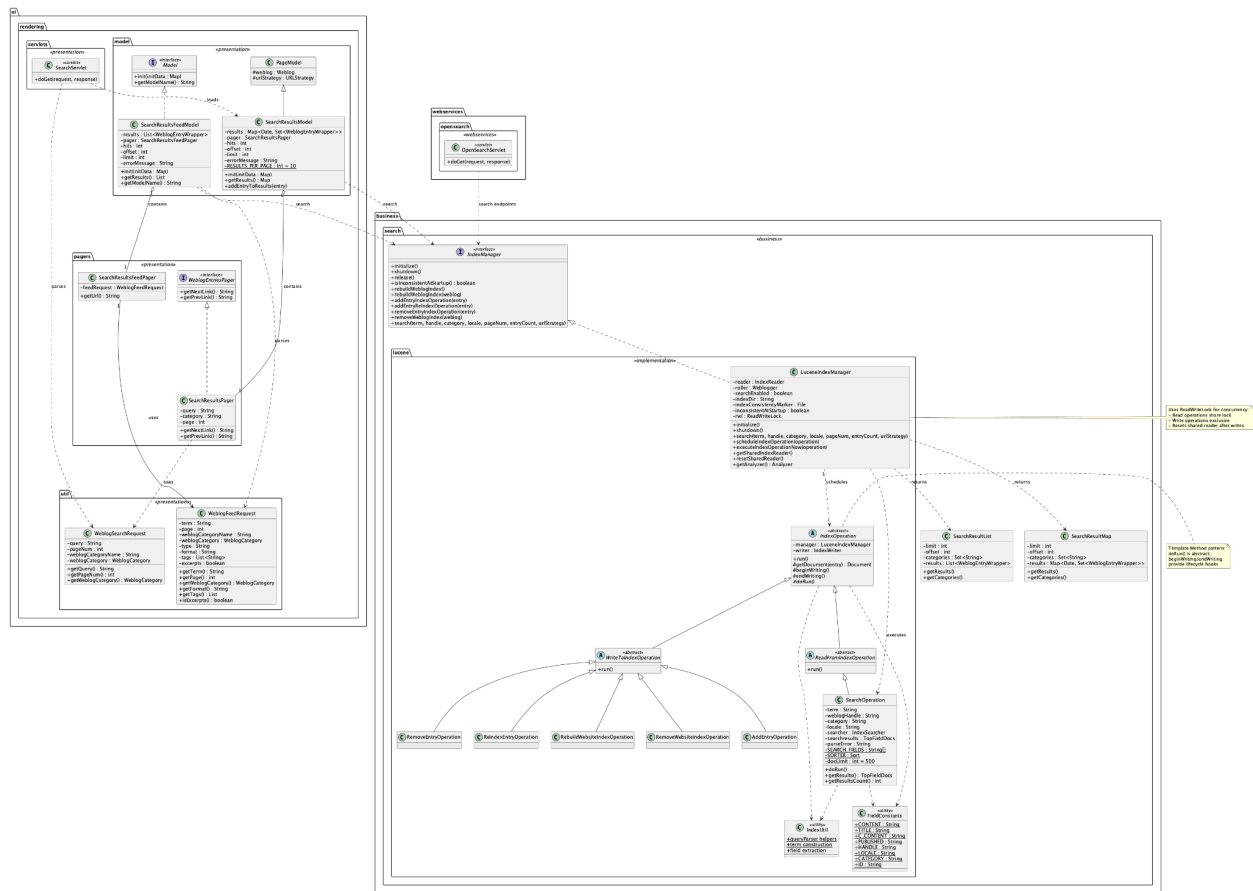
Configuration Dependencies

The search and indexing subsystem relies on several configuration properties:

- **`search.enabled`** - Feature toggle to enable/disable search functionality (default: true)
- **`search.index.dir`** - File system path to Lucene index directory (default: `${roller.data.dir}/search-index`)
- **`lucene.analyzer.name`** - Fully qualified class name of Lucene Analyzer to use for text processing (e.g., `org.apache.lucene.analysis.standard.StandardAnalyzer`)
- **`lucene.analyzer.maxTokenCount`** - Maximum number of tokens to analyze per field (default: 10000, prevents memory issues with very large documents)
- **`site.newsfeeds.defaultEntries`** - Default number of entries to return in feed-based searches (used by `SearchResultsFeedModel`)

These properties are accessed through `WebloggerConfig` and `WebloggerRuntimeConfig` and affect index initialization, query execution, and result formatting.

UML Diagram:



Design Patterns Employed

The search and indexing subsystem demonstrates several design patterns:

1. Template Method Pattern

`IndexOperation.doRun()` defines the template for all index operations. Concrete subclasses implement `doRun()` while the base class manages the lifecycle through `beginWriting()` and `endWriting()` hooks that bracket index modifications.

2. Strategy Pattern

`ReadFromIndexOperation` vs `WriteToIndexOperation` provide different concurrency strategies. Read operations acquire read locks and share the `IndexReader`, while write operations acquire exclusive write locks and reset the shared reader.

3. Service Locator Pattern

`WebloggerFactory` is used throughout for service discovery. While this simplifies access to services like `WeblogEntryManager` and `WeblogManager`, it's an anti-pattern that makes dependencies implicit and reduces testability.

4. **Decorator Pattern**

`LimitTokenCountAnalyzer` wraps the configured base analyzer (e.g., `StandardAnalyzer`) to enforce the `maxTokenCount` limit, preventing memory issues with extremely large documents.

5. **Front Controller Pattern**

`SearchServlet` acts as the front controller for all HTML search requests, delegating to models and renderers.

6. **Model-View-Controller (MVC)**

Clear separation between `SearchServlet` (controller), `SearchResultsModel/SearchResultsFeedModel` (model), and Velocity templates (view).

7. **Data Transfer Object (DTO)**

`WeblogSearchRequest` and `WeblogFeedRequest` encapsulate request parameters and validation logic, acting as DTOs between the presentation and business layers.

Design Strengths and Weaknesses

Strengths

1. **Clear API vs implementation separation**

`IndexManager` defines a clean contract, while `LuceneIndexManager` encapsulates Lucene-specific details.

2. **Explicit operation hierarchy**

The `IndexOperation` + `Read/Write` subclasses form a clear concurrency model for search tasks.

3. **Dedicated request parsing**

`WeblogSearchRequest` and `WeblogFeedRequest` encapsulate parsing and validation of search parameters before invoking the indexing layer.

4. **Template Method pattern in `IndexOperation`**

Provides consistent lifecycle management (begin/end) while allowing operation-specific logic in subclasses.

5. **Read/write lock concurrency control**

Allows multiple concurrent searches while ensuring write operations have exclusive access, balancing performance and consistency.

Weaknesses

1. **Service Locator coupling**

`LuceneIndexManager` and search operations frequently access services via `WebloggerFactory`, which makes dependencies implicit and harder to test. Dependency injection would be preferable.

2. **Large responsibility surface in `LuceneIndexManager`**

It handles lifecycle, scheduling, query execution, and result conversion, which may be too much for one class. Could benefit from separating concerns (e.g., separate scheduler, searcher, converter classes).

3. **Tight dependency on Lucene data structures**

The conversion logic (`convertHitsToEntryList`) directly manipulates Lucene `Document` fields and search results, creating strong coupling to Lucene APIs.

4. **Duplicated paging logic**

Search paging is implemented separately for HTML (`SearchResultsPager`) and feeds (`SearchResultsFeedPager`), which creates parallel logic and maintenance overhead.

5. **Hard-coded docLimit constraint**

`SearchOperation` has a fixed `docLimit = 500`, which cannot be configured per-query or globally. Large result sets are silently truncated.

6. **Fixed search field configuration**

The `SEARCH_FIELDS` array is static (`CONTENT`, `TITLE`, `C_CONTENT` only), preventing runtime configuration or custom field searches.

7. **Fragile inconsistency detection**

Using a file system marker file (`indexConsistencyMarker`) to track index state is fragile—file deletion or permission issues could trigger unnecessary rebuilds.

8. **No pagination within SearchOperation**

`SearchOperation` fetches up to 500 documents and relies on external pagination logic in models, creating inefficiency when only a few results are needed.

Manual vs LLM-Based Analysis (Small Slice)

LLM used: Claude (failed)

LuceneIndexManager.java
JAVA

SearchResultList.java
52 lines
JAVA

IndexUtil.java
71 lines
JAVA

SearchOperation.java
230 lines
JAVA

FieldConstants.java
47 lines
JAVA

Analyze a small slice of Apache Roller's Search & Indexing subsystem (Lucene path), for Task 1 "Manual vs LLM" comparison.
Scope: Query construction and execution only (not full subsystem).
Files to analyze (and only these):

- * SearchOperation.java
- * LuceneIndexManager.java
- * IndexUtil.java
- * FieldConstants.java
- * SearchResultList.java

create a uml diagram and class description for the same

10:44 PM  

 Pondering, stand by...

LLM used: Chatgpt thinking (gpt 5.2)

◆ Get Plus x



SearchResultList.java

File



FieldConstants.java

File



IndexUtil.java

File



LuceneIndexManager.java

File



SearchOperation.java

File

Analyze a small slice of Apache Roller's Search & Indexing subsystem (Lucene path), for Task 1 "Manual vs LLM" comparison.

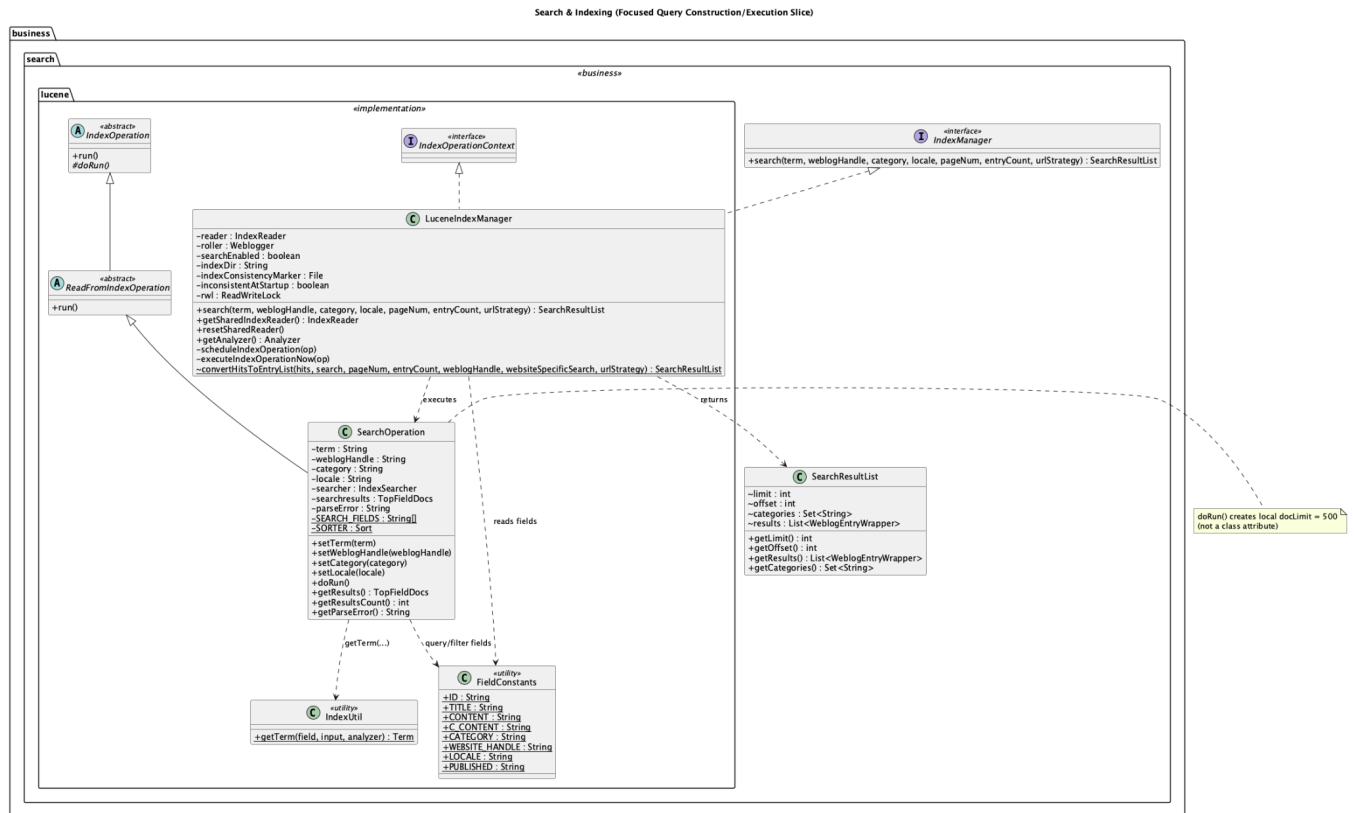
Scope: Query construction and execution only (not full subsystem).

Files to analyze (and only these):

- * SearchOperation.java
- * LuceneIndexManager.java
- * IndexUtil.java
- * FieldConstants.java
- * SearchResultList.java

create a uml diagram and class description for the same





Subsystem analyzed:

Search and indexing

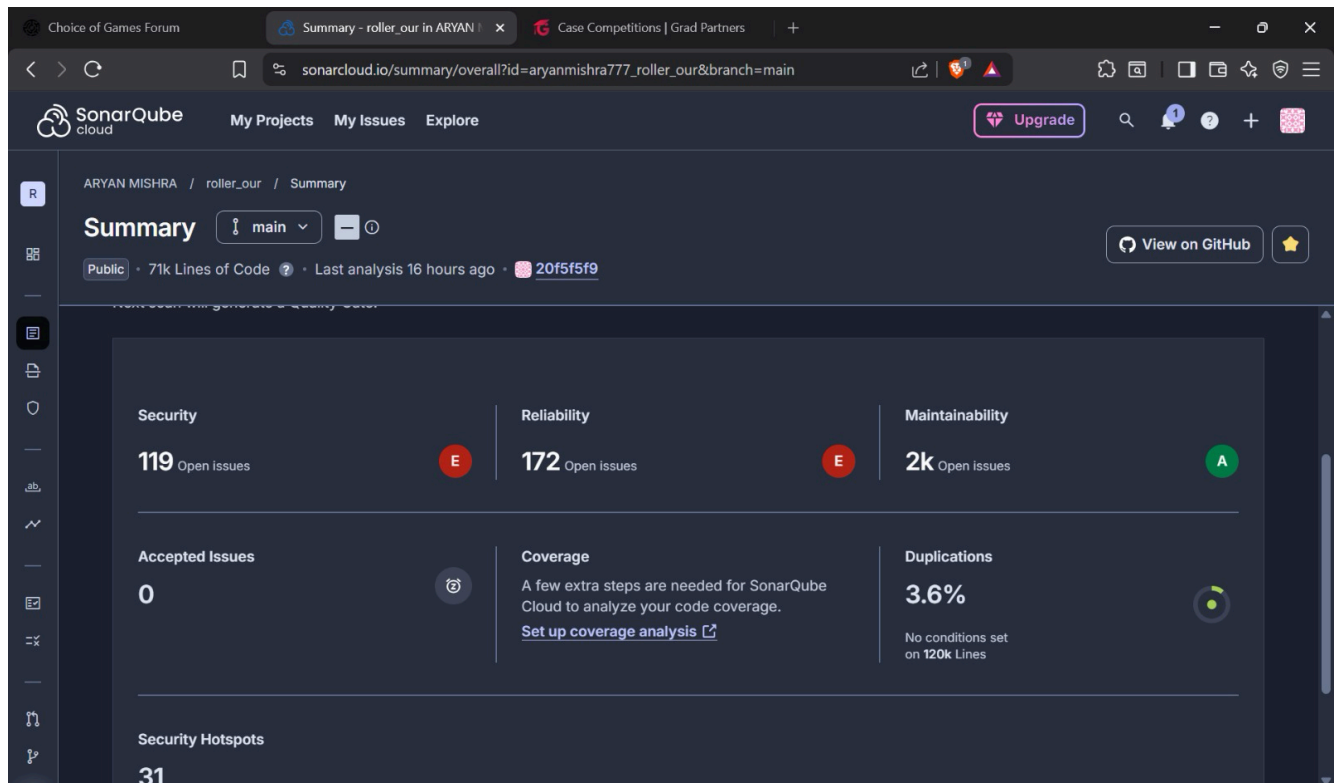
Comparative Analysis: Manual vs LLM-Assisted Understanding

- **Completeness:** The LLM provided a broader architectural view, while the manual analysis called out concrete behaviors (fixed `docLimit`, fixed field list).
- **Correctness:** Both were correct, but the manual analysis was more precise about code-level constraints.
- **Effort and Time:** The LLM was faster for an initial summary, but manual inspection was needed to confirm specifics.

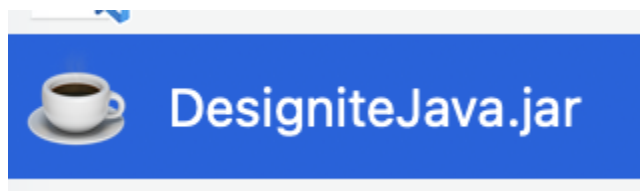
Usefulness for Design Reasoning: The LLM summary was useful for system-level context and was much faster.

Task 2

Task 2A: Design Smells



- For design smells we used sonarqube supplemented with designite.



We identified around ~400 smells on designite and ~200 smells using sonarqube. The issues we felt were the most important were:

1. Insufficient Modularization (God Class) in dateUtil

DateUtil is a God utility class with WMC=69 and 49 public methods — the highest Weighted Methods per Class value among all utility classes. It has 4 SonarQube issues and multiple Designite smells.

Evidence

SonarQube (4 issues)

- S1192 (String Literal Duplication): Repeated date format strings
- S1135 (TODO markers)

Designite

- Insufficient Modularization: WMC=69 with 49 methods — far exceeds reasonable thresholds (typical max WMC ~20-30)
- Deficient Encapsulation: 10 fields with 1 public field; utility methods are all static but class exposes internal state
- Implementation Smells:
 - parseWeblogURLDateString() — 2 Complex Conditionals + 2 Magic Numbers
 - getNoonOfDay() — Magic Number
 - formatIso8601() — 2 Magic Numbers
- Type Metrics: NOF=10, NOPF=1, NOM=49, NOPM=49, LOC=353, FANIN=19, FANOUT=2

2. Cyclic Dependency & Broken Hierarchy in `business.search.lucene`

The `business.search.lucene` package exhibits a severe **Cyclic Dependency** among its core classes, creating a "tangled" architecture where high-level managers and low-level operations are inextricably linked. This violates the **Acyclic Dependencies Principle (ADP)**.

The package suffers from a **Broken Hierarchy** (violating the **Liskov Substitution Principle**).

Evidence

Designite Java Analysis

- **Cyclic-Dependent Modularization (4-Way Cycle):** The following classes form a closed loop of dependencies: `LuceneIndexManager` ↔ `SearchOperation` ↔ `IndexOperation` ↔ `IndexUtil`
- **Broken Hierarchy (5 Classes):** The following classes are flagged for violating parent contracts (Liskov Substitution Principle):
 - `RemoveEntryOperation`
 - `AddEntryOperation`
 - `SearchOperation`
 - `RebuildWebsiteIndexOperation`
 - `RemoveWebsiteIndexOperation`

3. Hub-like Modularization (God Object) in WeblogEntry Class

The WeblogEntry class is a **Hub-like God Object**, acting as a central coupling point for the entire system with excessive dependencies (Fan-Out = 16). Instead of functioning as a simple Domain Object (POJO), it has accumulated too many responsibilities, resulting in a complex, fragile class that is hard to maintain and test.

Supporting Evidence

Designite Java Analysis

- **Hub-like Modularization:** Flagged for excessive **Fan-Out (16)**, indicating it depends on too many other classes.
- **Cyclic-Dependent Modularization:** Involved in a dependency cycle with other POJO classes.
- **Insufficient Modularization:** Extremely high Weighted Methods per Class (WMC), showing it handles too many distinct tasks.
- **Deficient Encapsulation:** Internal state is publicly exposed.

SonarQube & Implementation Issues

- **S3776 (Cognitive Complexity):** Multiple methods exceed complexity limits.
- **S1948 (Serialization):** Contains non-serializable fields in a serializable class.
- **S1192 (Duplication):** Widespread string literal duplication.

4. Insufficient Modularization (God Class) in the JPAWeblogEntryManagerImpl Class

The JPAWeblogEntryManagerImpl class exhibits **Insufficient Modularization**, functioning as a **God Class** for the persistence layer. It aggregates too many distinct responsibilities—managing weblog entries, comments, categories, tags, and hit counts—into a single, monolithic manager. This violates the **Single Responsibility Principle (SRP)**, resulting in a class with excessive complexity and low cohesion.

Supporting Evidence

Designite Java Analysis

- **Insufficient Modularization:** The class is flagged for having too many methods and responsibilities. It centralizes logic that should likely be split into separate managers (e.g., CommentManager, TagManager, CategoryManager).

- **Complex Methods:** High complexity flagged in `saveWeblogEntry()`, `getNextPrevEntries()`, `getWeblogEntries()`, `getComments()`, and `getTags()`.
- **Long Parameter Lists:** Found in `getNextPrevEntries()`, `removeMatchingComments()`, `getMostCommentedWeblogEntries()`, and `getTags()`.
- **Magic Numbers:** Over **20+ instances** detected across methods like `getMostCommentedWeblogEntries(6)`, `getPopularTags(5)`, and `applyCommentDefaultsToEntries(2)`.
- **Empty Catch Clause:** Found in `resetHitCount()`.
- **Long Identifier:** `getMostCommentedWeblogEntries`.

SonarQube (12 Issues)

- **S3776 (Cognitive Complexity):** Multiple query-building methods exceed maintainable complexity thresholds.
- **S1192 (String Literal Duplication):** Repeated JPQL query fragments indicating a lack of query externalization or constant usage.
- **S1135 (TODO Markers):** Presence of unfinished tasks.
- **S108 (Empty Catch Blocks):** Swallowed exceptions, risking hidden failures in the persistence layer.

5. Complex Methods & Long Parameter Lists in MetaWeblogAPIHandler Class

The `MetaWeblogAPIHandler` class serves as the XML-RPC entry point but suffers from severe complexity issues, ranking **#3 overall** in the codebase for smell count with **31 SonarQube issues**. The core problem lies in its handler methods (`editPost`, `newPost`), which are overly complex and require excessive parameters, making the API layer fragile and difficult to test or extend.

Supporting Evidence

SonarQube Analysis (31 Issues)

- **S3776 (Cognitive Complexity):** Multiple methods exhibit excessive complexity, making the logic hard to follow.
- **S1192 (String Literal Duplication):** Widespread repetition of XML-RPC string literals (e.g., parameter keys).
- **S112 (Generic Exceptions):** Methods throw generic `Exception`, masking specific failure modes.
- **S1186 & S1135:** Presence of empty methods and TODO markers, indicating unfinished or "dead" code.

- **Complex Method:** `editPost()` and `newPost()` are explicitly flagged for high cyclomatic complexity.
- **Long Parameter List:** Both `editPost()` and `newPost()` are flagged for having excessive parameters (e.g., passing username, password, blogid, struct, publish flag, etc., individually).
- **Magic Numbers:** `newPost()` contains hardcoded numeric literals (likely status codes).
- **Unutilized Abstraction:** The class structure may be over-abstracted relative to its actual usage.

6. Hub-like Modularization (God Object) & Cyclic Dependency in Weblog Class

The Weblog class is one of only **two classes** in the entire codebase flagged with **Hub-like Modularization**. It acts as a massive "God Object" in the POJO layer, maintaining connections to nearly every other domain object (Fan-Out). This central coupling point creates a rigid, tangled domain model where Weblog depends on entities that also depend on it (Cyclic Dependency).

Supporting Evidence

- **Hub-like Modularization:** Flagged for **Excessive Fan-Out**, indicating it connects to a disproportionately large number of other classes.
- **Cyclic-Dependent Modularization:** It is part of a dependency cycle involving WeblogEntry, WeblogCategory, WeblogBookmark, and others.
- **Insufficient Modularization:** The class has too many fields and methods for a single entity, attempting to model too much.
- **Deficient Encapsulation:** Internal state is exposed publicly.

- **S3776 (Cognitive Complexity):** Methods contain logic that exceeds complexity thresholds.
 - **S1948 (Serialization):** Non-serializable fields in a Serializable class.
-

Task 2B: Code Metrics Analysis

- For code metrics we used checkstyle, PMD, and CK metrics.

Selected Metrics

1. Cyclomatic Complexity (CC, method-level average)
2. Path Complexity (PC, method-level average; NPath-style path growth indicator)
3. Lines of Code (LOC, class-level average)
4. Class Coupling (FANOUT, class-level average)
5. Depth of Inheritance Tree (DIT, class-level average)
6. Weighted Methods per Class (WMC, class-level average)

Metrics values

Metric	Pre-refactor snapshot
Avg Cyclomatic Complexity (CC)	1.673
Avg Path Complexity (PC)	0.793
Avg LOC per Class	93.817
Avg Class Coupling (FANOUT)	3.631
Avg DIT	0.695
Avg WMC	16.474
Total Code Smells (Design + Implementation)	1,436

Task 3: Refactoring

Task 3A: Manual Refactoring

- Issues added and signed
- Fixed in separate branches and then merged using PRs

is:issue state:closed

✕

🔍

🏷 Labels

📌 Milestones

New issue

☐

Open

4

Closed

6

Author

Labels

Projects

Milestones

Assignees

Types

📄 Newest

☐

🔄

Refactor Weblog — Hub-like God Object with Cyclic Dependencies

refactoring

#11 · by VA24d was closed 2 days ago

☐

🔄

Refactor DateUtil — God Utility Class with WMC=69

refactoring

#10 · by VA24d was closed 2 days ago

🔗 1

☐

🔄

Break Cyclic Dependencies in search.lucene Package

refactoring

#9 · by VA24d was closed 2 days ago

🔗 1

💬 1

☐

🔄

Refactor JPAWeblogEntryManagerImpl — Insufficient Modularization God Class

refactoring

#8 · by VA24d was closed 2 days ago

☐

🔄

Refactor MetaWeblogAPIHandler — Complex Methods with Long Parameter Lists

refactoring

#5 · by VA24d was closed yesterday

💬 1

☐

🔄

Refactor WeblogEntry — Hub-like God Object in Domain Layer

refactoring

#4 · by VA24d was closed 2 days ago

💬 1

Task 3B: Post-Refactoring Metrics

Metric Implications and Refactoring Decisions

Cyclomatic Complexity improved (1.673 → 1.658): indicates slightly simpler control flow per method.

Path Complexity improved (0.793 → 0.787): indicates lower path growth and easier test-path reasoning.

Avg LOC per class improved (93.817 → 93.429): suggests mild size reduction and better decomposition.

Class Coupling improved (3.631 → 3.607): suggests reduced dependency burden and change ripple risk.

Avg DIT improved (0.695 → 0.683): inheritance hierarchy is slightly flatter, aiding maintainability.

Avg WMC improved (16.474 → 16.435): class-level complexity reduced incrementally.

Total code smells improved (1,436 → 1,429): net reduction of 7 smell instances after refactoring.

Decision guidance: prioritize by combined score = smell frequency + PMD severity + WMC percentile.

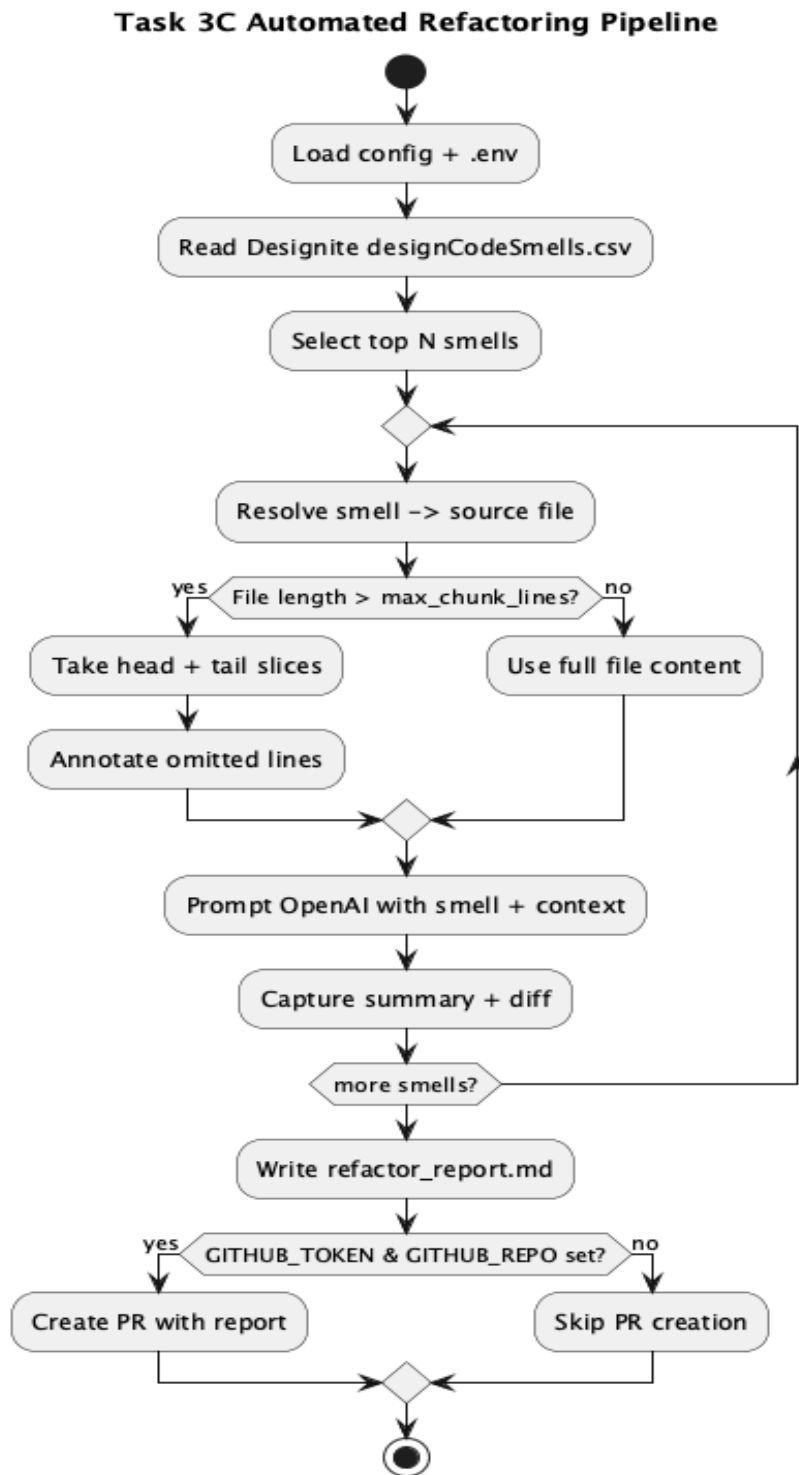
Refactoring Impact Metrics

Metric	Pre-refactor	Post-refactor	Delta	Interpretation
Total Code Smells (Design + Implementation)	1,436	1,429	-7	Net smell reduction
Avg WMC	16.474	16.435	-0.039	Slightly lower class complexity
Avg CC	1.673	1.658	-0.015	Slightly simpler control flow
Avg PC	0.793	0.787	-0.006	Lower path complexity
Avg FANOUT	3.631	3.607	-0.024	Lower coupling
Avg DIT	0.695	0.683	-0.012	Flatter inheritance hierarchy

Additional Stability Indicators

Indicator	Pre-refactor	Post-refactor	Delta
Methods with CC > 10	53	53	0

Task 3C: Automating the Refactoring Pipeline using LLMs



chore(task3): LLM refactoring suggestions #2

 Merged Hardik1106 merged 10 commits into `master` from `vijay` 2 days ago

 Conversation 0

 Commits 10

 Checks 12

 Files changed 78



VA24d commented [last week](#)

Task 3C LLM Refactoring Suggestions

This PR is **auto-generated** by the Task 3C pipeline. No source code changes are committed; the report is provided for review and documentation.

Summary

Task 3C LLM Refactoring Report

Generated: 2026-02-05 10:04:47 UTC

Detected Design Smells

The pipeline code in repo `/project-1-team-33/task3-pipeline`
PR created (merged by mistake before being reverted)

Task 4: Agentic Refactoring on a Single File

Note: All of the prompts and outputs for this task have been committed to **aditya-task-4** branch (Not merged). The code given by the AI can thus be accessed from the same branch.

Environment configuration used for Agentic refactoring

- **Editor:** Visual Studio Code
- **Agentic Tool:** GitHub Copilot
- **Mode:** Agent
- **AI Model Used:** Claude Opus 4.6 😊
- **Prompt Format Used:** Given below and in file *prompt.md* as committed to **aditya-task-4** branch of the repo (Not merged). To be used by replacing [THESE BLOCKS] with the design smell and file names that need to be addressed.
- The agent would need user permission and approval to go ahead with certain tasks.
- File addressed for Task 4: *MetaWeblogAPIHandler.java*

Prompt used for the AI Agent:

You are an expert coding agent in Java, working on a project to analyse and identify designated design smell within the codebase, design the solution for it and refactor the code based on it, ensuring that the functionality of the code remains completely unaffected and no new design smell is introduced in the process.

You have access to the entire repository using context tools. If you need to check helper classes or parent classes to verify a refactoring, please do so during the Analysis phase.

I will be providing you with the filename(s) and the output of static analysis tools I have used, and I need you to address the given smell and refactor only within the context of the smell.

Broadly I need you to ensure all 3 stages are taken care of in the process of this refactor:

1. Smell Identification (Analysis)

- Take a single Java file
- Optionally use static analysis output
- Identifies and select design-level smells to address

2. Planning

- Propose refactoring strategies for selected smells
- Briefly explain why the refactoring is appropriate
- Identify affected methods/classes

3. Refactoring Execution

- Based on the above outputs, once I say proceed, you will go according to plan and generate the full refactored code/code snippets as necessary
- Ensure syntactic correctness/compilability
- Clearly distinguish original vs refactored code

And then when the refactoring is approved by me, you may refactor the codebase, but do not do that until I approve it.

This is the file I need you to work on: [FILENAME], and this file contains the static analysis output: [STATIC_ANALYSIS_FILE]

Agent Output:

```
✓ Searched codebase for design smells and planned refactoring
✓ Reviewed MetaWeblogAPIHandler.java and analyzed static analysis data

• The user wants me to analyze the MetaWeblogAPIHandler.java file for design smells, plan refactoring, and then execute it after approval. Let me first read the full file and the relevant static analysis data to understand the situation.
  Let me start by reading the full MetaWeblogAPIHandler.java file and checking the static analysis outputs for this specific class.

I'll start by gathering the full context needed for the analysis. Let me read the complete source file and filter the relevant static analysis data.

✓ Reviewed MetaWeblogAPIHandler.java and searched for terms

📄 Read MetaWeblogAPIHandler.java
🔍 Searched for text MetaWeblogAPIHandler (**/analysis-pre-refactor 10-02-2026/designite/*.csv), 20 results
🔍 Searched for text BloggerAPIHandler (**/analysis-pre-refactor 10-02-2026/designite/*.csv), 20 results

Now let me read the parent class BloggerAPIHandler to understand the inheritance relationship, which is central to the "Broken Hierarchy" smell detected on the parent.

✓ Searched for files and reviewed multiple Java files

🔍 Searched for files matching **/BloggerAPIHandler.java, 1 match
🔍 Searched for text MetaWeblogAPIHandler (**/*.java), 9 results
📄 Read BloggerAPIHandler.java, lines 1 to 400
📄 Read BloggerAPIHandler.java, lines 400 to 501
📄 Read BaseAPIHandler.java, lines 1 to 120
📄 Read BaseAPIHandler.java, lines 120 to 203

• Now I have the full picture. Let me also check the implementation code smells for MetaWeblogAPIHandler to get the complete picture.
  Let me look at the relevant type metrics headers to understand the columns:

Now let me check the type metrics header to interpret the columns correctly:

✓ Reviewed 2 files

📄 Read typeMetrics.csv, lines 1 to 2
📄 Read implementationCodeSmells.csv, lines 1 to 3

I now have all the context needed. Here is my complete analysis and plan.
```

Code:

All of the prompts and outputs for this task have been committed to **aditya-task-4** branch (Not merged). The code given by the AI can thus be accessed from the same branch.

Summary of changes

Summary of Changes (Original vs Refactored)				
#	Location	Original	Refactored	Rationale
1	Line 58 — class declaration	<code>extends BloggerAPIHandler</code>	<code>extends BaseAPIHandler</code>	Fixes Unutilized Abstraction : <code>MetaWeblogAPIHandler</code> never used any <code>BloggerAPIHandler</code> -specific behavior; all inherited utilities (<code>validate()</code> , <code>flushPageCache()</code> , constants) come from <code>BaseAPIHandler</code>
2	<code>editPost</code> line 148–150	Inline <code>dateCreated / pubDate</code> extraction (3 lines)	<code>extractDateCreated(postcontent)</code> call	Eliminates duplication with <code>newPost</code> ; reduces CC
3	<code>editPost</code> line 169–172	Inline <code>if (publish) { setStatus(PUBLISHED) } else { setStatus(DRAFT) }</code>	<code>applyPublishStatus(entry, publish)</code> call	Eliminates duplication with <code>newPost</code> ; reduces CC
4	<code>newPost</code> line 246–249	Inline <code>dateCreated / pubDate</code> extraction (3 lines)	<code>extractDateCreated(postcontent)</code> call	Same as #2
5	<code>newPost</code> line 271–275	Inline publish status if/else	<code>applyPublishStatus(entry, publish)</code> call	Same as #3
6	New method	—	<code>applyPublishStatus(WeblogEntry, boolean)</code>	Extracted helper for publish/draft logic
7	New method	—	<code>extractDateCreated(Hashtable)</code>	Extracted helper for date extraction with fallback

What is preserved

- All **public method signatures** are identical (XML-RPC contract unchanged)
- All **functional behavior** is identical — validation, entry creation/editing, flushing, media upload, recent posts
- `BloggerAPIHandler` is **not modified**
- No new design smells introduced; the hierarchy is flattened correctly since `BaseAPIHandler` provides everything needed

Task 5: Comparative Refactoring Analysis (Manual vs LLM vs Agentic)

All of the prompts and outputs for this task have been committed to **aditya-task-4** branch (Not merged). The code given by the AI can also, thus, be accessed from the same branch.

1. MetaWeblogAPIHandler.java (Issue 5 refactoring)

File addressed: MetaWeblogAPIHandler.java (Issue #5 in the repo)

Smell Addressed: Complex Methods / Unutilized Abstraction / Long Parameter Lists

Changes made by Agentic AI:

Summary of Changes (Original vs Refactored)				
#	Location	Original	Refactored	Rationale
1	Line 58 — class declaration	<code>extends BloggerAPIHandler</code>	<code>extends BaseAPIHandler</code>	Fixes Unused Abstraction : <code>MetaWeblogAPIHandler</code> never used any <code>BloggerAPIHandler</code> -specific behavior; all inherited utilities (<code>validate()</code> , <code>flushPageCache()</code> , constants) come from <code>BaseAPIHandler</code>
2	<code>editPost</code> line 148–150	Inline <code>dateCreated / pubDate</code> extraction (3 lines)	<code>extractDateCreated(postcontent)</code> call	Eliminates duplication with <code>newPost</code> ; reduces CC
3	<code>editPost</code> line 169–172	Inline <code>if (publish) { setStatus(PUBLISHED) } else { setStatus(DRAFT) }</code>	<code>applyPublishStatus(entry, publish)</code> call	Eliminates duplication with <code>newPost</code> ; reduces CC
4	<code>newPost</code> line 246–249	Inline <code>dateCreated / pubDate</code> extraction (3 lines)	<code>extractDateCreated(postcontent)</code> call	Same as #2
5	<code>newPost</code> line 271–275	Inline publish status if/else	<code>applyPublishStatus(entry, publish)</code> call	Same as #3
6	New method	—	<code>applyPublishStatus(WeblogEntry, boolean)</code>	Extracted helper for publish/draft logic
7	New method	—	<code>extractDateCreated(Hashtable)</code>	Extracted helper for date extraction with fallback

What is preserved

- All **public method signatures** are identical (XML-RPC contract unchanged)
- All **functional behavior** is identical — validation, entry creation/editing, flushing, media upload, recent posts
- `BloggerAPIHandler` is **not modified**
- No new design smells introduced; the hierarchy is flattened correctly since `BaseAPIHandler` provides everything needed

Prompt used for LLM:

LLM used: Gemini Pro 3 (in Thinking mode)

You are an expert programmer in Java, working on a project to analyse and identify designated design smell within the codebase, design the solution for it and refactor the code based on it, ensuring that the functionality of the code remains completely unaffected and no new design smell is introduced in the process.

As of now we are working on the code from a version of Apache's Roller codebase.

I will be providing you with the file(s) and the output of static analysis tools I have used, and I need you to address the given smell and refactor only within the context of the smell.

Broadly I need you to ensure all 3 stages are taken care of in the process of this refactor:

1. Smell Identification (Analysis)

- Take a single Java file
- Optionally use static analysis output
- Identifies and select design-level smells to address

2. Planning

- Propose refactoring strategies for selected smells
- Briefly explain why the refactoring is appropriate

- Identify affected methods/classes

3. Refactoring Execution

- Based on the above outputs, you will then go according to plan and generate the full refactored code/code snippets as necessary
- Ensure syntactic correctness/compilability
- Clearly distinguish original vs refactored code

Do all of this in one shot, I will not be giving you further prompts. Along with your analysis also give me the final changed code as well as a summary of changes.

This is the file I need you to work on: `MetaWeblogAPIHandler.java`, and this file contains the static analysis output from designite: `designCodeSmells.csv`.

All relevant code and analysis output is given below.

`MetaWeblogAPIHandler.java`:

[entire file contents]

`designCodeSmells.csv`:

[entire file contents]

Summary of changes by LLM output:

Summary of Changes

- 1. Duplicate Code Removed:** Extracted the `saveAndFlushEntry` method. This method consolidates the calls to `saveWeblogEntry`, `flush`, `flushPageCache`, and `Thread.sleep` which were previously duplicated in `editPost` and `newPost`.
- 2. Magic Strings Replaced:** Introduced `private static final String` constants (e.g., `KEY_TITLE`, `KEY_DESCRIPTION`) for all XML-RPC structure keys. This prevents typos and centralizes key definitions, improving the "Abstraction" by defining its data contract clearly.
- 3. Refactored Date Parsing:** Created `parseDateCreated` to handle the check for `dateCreated` versus `pubDate`, removing minor logic duplication.
- 4. Preserved API Contract:** No public method signatures or logic flows were altered; the class functions exactly as before but is structurally cleaner.

Comparative Refactoring Analysis

1. Clarity

- **Manual (Ref. Issue #5):** Superior. By introducing specific Domain classes like `NewPostRequest` and `MediaUploadRequest`, the manual approach transformed cryptic XML-RPC mappings into readable Java objects. It effectively "typed" the untyped `Hashtable` input.
- **LLM (Gemini One-Shot):** Moderate. It improved readability significantly by extracting magic strings (`KEY_TITLE`) and consolidating the persistence logic into `saveAndFlushEntry`.
- **Agentic (Claude Agent):** Mixed. It improved structural clarity by removing the misleading inheritance, but it failed to extract magic strings, leaving the code cluttered with "title" and "description" literals, making the business logic harder to scan than the LLM version.

2. Conciseness

- **Manual:** Prioritized separation of concerns over raw LOC. By extracting 4 new helper classes (`PostRequestAdapter`, etc.), it distributed complexity rather than just compressing it.

- **LLM (Gemini One-Shot):** Most Concise (Locally). It successfully identified a 15-line block of exact code duplication (persistence + sleep) and replaced it with a single method call *saveAndFlushEntry*, reducing the file size immediately.
- **Agentic (Claude Agent):** Similar to LLM in extraction, but missed the opportunity to deduplicate the largest chunk of logic (the save/flush sequence), focusing instead on smaller helpers like *applyPublishStatus*.

3. Design Quality (DRY & SOLID)

- **Manual:** Best Adherence to SOLID.
The manual refactor explicitly added "constructor-based dependency injection" to fix the tight coupling to *WebloggerFactory*.
It proactively replaced legacy Hashtable with HashMap and fixed a resource leak (try-with-resources), which neither AI attempted.
- **LLM (Gemini One-Shot):** Good DRY adherence (extracted constants/methods) but missed the architecture. It diagnosed "Unutilized Abstraction" but implemented a fix that didn't address the abstraction at all, only the internal code quality.
- **Agentic (Claude Agent):** Best Hierarchy Design. It was the only approach to correctly identify that *MetaWeblogAPIHandler* had a fake inheritance relationship with *BloggerAPIHandler*. By changing the parent to *BaseAPIHandler*, it fixed the underlying OOD violation (Liskov Substitution/Interface barriers).

4. Faithfulness

- **LLM & Agent:** Strictly Faithful (To a Fault). Both AI models saw *Thread.sleep(1000)* and Hashtable and preserved them perfectly. They interpreted "Refactoring" as "Change structure, keep exact behavior."
- **Manual:** Pragmatic Unfaithfulness. The manual team recognized *Thread.sleep(1000)* as a performance bug (throttling) and removed it. Also incorrectly/boldly migrated logic to use modern Collections. Humans improved the system's runtime behavior; AIs merely tidied up the slow behavior.

5. Architectural Impact

- **Manual:** High. Made the handler unit-testable (via DI) and modernized the API surface.
- **Agentic (Claude Agent):** Medium-High. Fixed the inheritance tree. If applied globally, this would untangle the messy web of API handlers in Roller.
- **LLM (Gemini One-Shot):** Low. Purely cosmetic cleanup inside one file.

6. Human vs. Automation Judgment

- **Where Human Refactoring Was Superior:**
Performance & Modernization: We identified that *Thread.sleep* was "slowing down every request" and legacy Hashtable was obsolete. The AIs, constrained by "Refactor only," treated these as sacred requirements.
Data Modeling: We saw a "Long Parameter List" and thought "This needs a Request Object (NewPostRequest)." The AIs saw "Long Parameter List" and just kept passing the Hashtable around.
- **Where Agentic Refactoring Was Advantageous:**
Inheritance Analysis: The Agent performed a cross-file analysis (reading *BloggerAPIHandler*) and realized the inheritance was unused. The Manual team fixed "Tight coupling to static factory" but the notes do not mention fixing the confusing inheritance tree.

- **Where Automation Failed:**
Agent: Overlooked the "Magic Strings" smell completely, despite it being a standard fix.
LLM: Correctly identified the Design Smell ("Unutilized Abstraction") in its text analysis but failed to generate code that actually fixed it.

Conclusion: The **Manual** refactoring was an upgrade (better performance, modern types, testability). The **Agent** was a correct architectural repair (inheritance). The **LLM** was a cleanup (constants/DRY). This highlights that AI tools currently struggle to make "Executive Decisions" that humans generally make easily.

2. WeblogEntry.java (Issue 4 refactoring)

File addressed: WeblogEntry.java (Issue #4 in the repo)
Smell Addressed: Hub-like Modularization (God Object)

Changes made by Agentic AI:

Changes Made

New file: WeblogEntryContentRenderer.java

A focused class (~220 LOC) in the same package that encapsulates all content rendering/display logic:

Method	Responsibility
render(String)	Plugin-based text transformation + HTML sanitization
getTransformedText()	Renders entry text through plugins
getTransformedSummary()	Renders entry summary through plugins
displayContent(String)	Selects text vs. summary + "Read More" link logic
getDisplayContent()	Convenience wrapper for displayContent(null)
getDisplayTitle()	Title fallback to truncated text
getRss09xDescription() / (int)	HTML-escaped RSS description
formatPubTime(String)	Locale-aware publish time formatting
formatUpdateTime(String)	Update time formatting

Modified file: WeblogEntry.java

- Added a transient WeblogEntryContentRenderer contentRenderer field with lazy initialization via getContentRenderer()
- All 10 methods above became **one-line delegation calls** (e.g., return getContentRenderer().formatPubTime(pattern);)
- The private render(String) method was **fully removed** (it now lives in the renderer)
- 6 imports removed:** SimpleDateFormat, Map, StringEscapeUtils, WeblogEntryPlugin, HTMLSanitizer, I18nMessages, RollerConstants
- LOC reduced:** 1031 → 946 (~85 lines removed)
- Public API unchanged** — all method signatures remain identical, preserving full backward compatibility with WeblogEntryWrapper and all 60+ consumers

Todos (4/4)

2 files changed +258 -110

WeblogEntry.java

project-1-team-33\app\src\main\java\org\apache\roller\weblogger\pojos

+24 -109

WeblogEntryContentRenderer.java

project-1-team-33\app\src\main\java\org\apache\roller\weblogger\pojos

+234 -1

Prompt used for LLM:

LLM used: Gemini Pro 3 (in Thinking mode)

You are an expert programmer in Java, working on a project to analyse and identify designated design smell within the codebase, design the solution for it and refactor the code based on it, ensuring that the functionality of the code remains completely unaffected and no new design smell is introduced in the process.

As of now we are working on the code from a version of Apache's Roller codebase.

I will be providing you with the file(s) and the output of static analysis tools I have used, and I need you to address the given smell and refactor only within the context of the smell.

Broadly I need you to ensure all 3 stages are taken care of in the process of this refactor:

- 1. Smell Identification (Analysis)
 - - Take a single Java file
 - - Optionally use static analysis output
 - - Identifies and select design-level smells to address
2. Planning
 - - Propose refactoring strategies for selected smells
 - - Briefly explain why the refactoring is appropriate
 - - Identify affected methods/classes
3. Refactoring Execution
 - - Based on the above outputs, you will then go according to plan and generate the full refactored code/code snippets as necessary
 - - Ensure syntactic correctness/compilability
 - - Clearly distinguish original vs refactored code

Do all of this in one shot, I will not be giving you further prompts. Along with your analysis also give me the final changed code as well as a summary of changes.

This is the file I need you to work on: WeblogEntry.java, and this file contains the static analysis output from designite: designCodeSmells.csv. Mainly the Hub-like God object one can see here being reported in the smells.

All relevant code and analysis output is given below.

WeblogEntry.java:
[entire file contents]

designCodeSmells.csv:
[entire file contents]

Summary of changes by LLM output:

Summary of Changes

1. **Creation of `WeblogEntryOperations`:** I created a private static inner class to house all logic that couples the `WeblogEntry` entity to external services (`WebloggerFactory`, `UserManager`) and configuration (`WebloggerConfig`). This class now acts as the implementation delegate for business operations.
2. **Delegation:** Existing methods in `WeblogEntry` that previously contained this logic (e.g., `getPermalink`, `render`, `getComments`, `hasWritePermissions`) were modified to delegate their execution to `WeblogEntryOperations`.
3. **Decoupling:** The main body of the `WeblogEntry` class is now focused primarily on its properties (getters/setters) and simple state management, significantly reducing its "Hub" nature. The complex interactions with the plugin system and URL strategies are physically separated within the file.
4. **No Public API Change:** All method signatures and return types remain identical, ensuring zero impact on the rest of the application.

Comparative Refactoring Analysis:

1. Clarity

- **Manual (Ref. Issue #4):** Superior Clarity. The manual team deconstructed the class into semantically meaningful concepts: *WeblogEntryPresenter* for UI logic, *TagManifest* for tag operations, and *PluginConfiguration* for parsing logic. Renaming confusing methods (e.g., *getRelativePermalink*) further reduced cognitive load.
- **LLM (Gemini One-Shot):** Poor Clarity. It utilized a confusing "private static inner class" pattern (*WeblogEntryOperations*) to hide complexity. While the main methods became delegations, the file remained a monolith, and the "Services inside a POJO" pattern is non-idiomatic and confusing for Java developers.
- **Agentic (Claude Agent):** Good Clarity. It correctly identified "Rendering" as the biggest clump of foreign logic and extracted it to *WeblogEntryContentRenderer*. This was a clean separation but less granular than the manual approach (which recognized Tags and Plugins as separate concerns too).

2. Conciseness

- **Manual:** While the total file count increased (4 new files), the *WeblogEntry* class itself became significantly leaner and focused purely on being an Entity. The introduction of the Builder pattern replaced a massive, error-prone constructor.
- **LLM (Gemini One-Shot):** Lowest Conciseness. It arguably increased verbosity by adding delegation boilerplate without actually reducing the file size or complexity. It just moved the mess into the basement (the inner class).
- **Agentic (Claude Agent):** Good reduction of the main class size. It stripped out the rendering methods effectively but left the Tag and Plugin logic mixed in, meaning the class was still larger than the Manual version.

3. Design Quality (DRY & SOLID)

- **Manual:** Best Adherence to SOLID.
SRP: Achieved multi-dimensional separation (Presentation, Configuration, Data Structure).
DIP: Crucially, it removed the dependency on the static *WebloggerFactory*, creating a testable entity that doesn't require a container.
Encapsulation: Fixed the long constructor smell via Builder pattern.

- **LLM (Gemini One-Shot):** Violated SOLID. It kept the tight coupling to *WebloggerFactory* (Services), just hiding it inside the inner class. It treated the symptom (file length) rather than the disease (coupling).
- **Agentic (Claude Agent):** Good adherence to SRP. It correctly decoupled dependencies like *HTMLSanitizer* and *WeblogEntryPlugin*. However, it missed the static service dependency issue that the manual team caught.

4. Faithfulness

- **All Approaches:** Preserved the public API signatures to maintain backward compatibility.
- **Manual:** Went a step further by actually fixing bugs (Serialization issue in attributes) and modernizing APIs (replacing Calendar with java.time), which technically changes implementation behavior but improves faithfulness to modern standards.

5. Architectural Impact

- **Manual:** High Positive Impact. By removing the Service Locator (*WebloggerFactory*) usage, the manual refactoring made the Domain layer unit-testable for the first time. The introduction of *TagManifest* suggests a move towards a richer Domain Model.
- **LLM (Gemini One-Shot):** Negative Impact. The "Service-in-Inner-Class" anti-pattern solidifies the hard dependency on the container, making the architecture more rigid.
- **Agentic (Claude Agent):** Moderate Positive Impact. It pushed the architecture towards a cleaner View/Model separation (Renderer pattern) but didn't solve the core architectural flaw of global service access.

6. Human vs. Automation Judgment

- **Where Human Refactoring Was Superior:**

Domain Insight: We recognized that "Tags" and "Plugins" were distinct domain concepts worthy of their own classes (*TagManifest*, *PluginConfiguration*). The AI just saw "methods to move."

Modernization: We proactively upgraded legacy APIs (Date -> java.time), which AI tools won't do unless explicitly asked.

Architectural Awareness: We realized the *WebloggerFactory* static dependency was the root cause of poor testability.
- **Where Agentic Refactoring Was Advantageous:**

Data-Driven Selection: The Agent correctly identified render() functions as the highest source of FANOUT complexity immediately. Its solution for that specific slice was nearly identical to the Manual "Presenter" solution.
- **Where Automation Failed:**

LLM: Proposed a "refactoring" that was purely syntactic (inner class) and ignored the architectural requirements of a rich domain model.

Agent: Missed the subtle serialization bugs and field visibility issues that the manual team caught via standard coding checklists.

Conclusion: The **Manual** refactoring was a comprehensive re-architecture that improved testability and modularity. The **Agent** correctly identified a major slice of the work (Rendering) but missed the finer grain. The **LLM** proved ineffective for architectural repair.

Contributions

- Vijay Aravynthan
 - Task 1
 - Search subsystem
 - Task 2
 - Designite
 - Code metrics
 - Task 3
 - Documentation
- Varun Patil
 - First subsystem of task 1 (weblog and content)
 - Ran sonarqube to find smells and code metrics
 - Helped select design/code smells to refactor
 - Assisted in task 3
 - Documentation
- Hardik Chadha
 - Task 1
 - User and Role Management Subsystem
 - Task 2
 - SonarQube
 - Documentation
- Aryan Mishra
 - Task 1
 - User and Role Management Subsystem
 - Task 2
 - SonarQube
 - Documentation
- Aditya Nair
 - Task 4 (entirely)
 - Task 5 (entirely)
 - Documentation

Done 🎉